© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_15

15. Does It Even Work? Testing and Evaluating LLM Apps

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to focus on how to test and evaluate large language model (LLM) applications.

Introduction

Enterprise software teams have long relied on unit and integration tests to validate deterministic application logic. However, when integrating LLMs into backend services and APIs, those familiar testing paradigms fall short. LLMs behave as probabilistic engines: the same prompt can yield different responses on each call, and models may "hallucinate," producing plausible-sounding but incorrect or unsafe content. As a result, achieving confidence in LLM-powered features requires a shift from code-centric tests to a comprehensive evaluation framework that treats model outputs as first-class artifacts to be tested.

This chapter introduces the principles and practices needed to evaluate and test LLM applications at scale. It will begin by defining the core quality dimensions—relevance, accuracy, completeness, clarity, style compliance, bias, safety, latency, and cost—that every LLM deployment must measure and guard. Next, you will survey the ecosystems of tools and libraries—from dedicated Python evaluation frameworks to Java-centric solutions such as Spring AI's Evaluator interface. Through concrete examples and code snippets, you'll learn how to decouple evaluation from generation, plug in specialized checks (including secondary LLMs for automated fact-checking), and bake these checks into your continuous integration/continuous development (CI/CD) pipelines. By the end of this chapter, you will have a clear roadmap for building repeatable, automated workflows that catch hallucinations, enforce policy rules, and ensure your LLM-enhanced applications remain robust, compliant, and cost-effective.

Evaluation of LLM Applications

As organizations embed large language models into customer-facing products, it becomes essential to verify their outputs before they reach end users. Without a systematic evaluation framework, models can hallucinate—asserting plausible-sounding but unfounded claims—drift off-topic by ignoring the user's actual intent or retrieved context, or they can inadvertently introduce biased or unsafe language. Rigorous evaluation catches these issues early, protects user trust, ensures compliance with style and safety requirements, and helps control compute costs by routing checks to appropriately sized models.

A flexible evaluation framework treats evaluation as a separate step from generation: it accepts the original user prompt, any contextual data supplied (for example, retrieved documents), and the model's generated response. Then it returns a pass/fail verdict or score. Within this framework, different evaluators can be plugged in—one might use an LLM to verify that an answer aligns with the user's question and the provided context; another might perform factual consistency checks against authoritative sources; others can screen for clarity, stylistic compliance, bias, toxicity, or any domain-specific rules. Because evaluation is decoupled, teams can choose smaller or more specialized models for each check, dramatically reducing latency and cost compared with running a large flagship model for every test. Figure 15-1 highlights a sample flow.

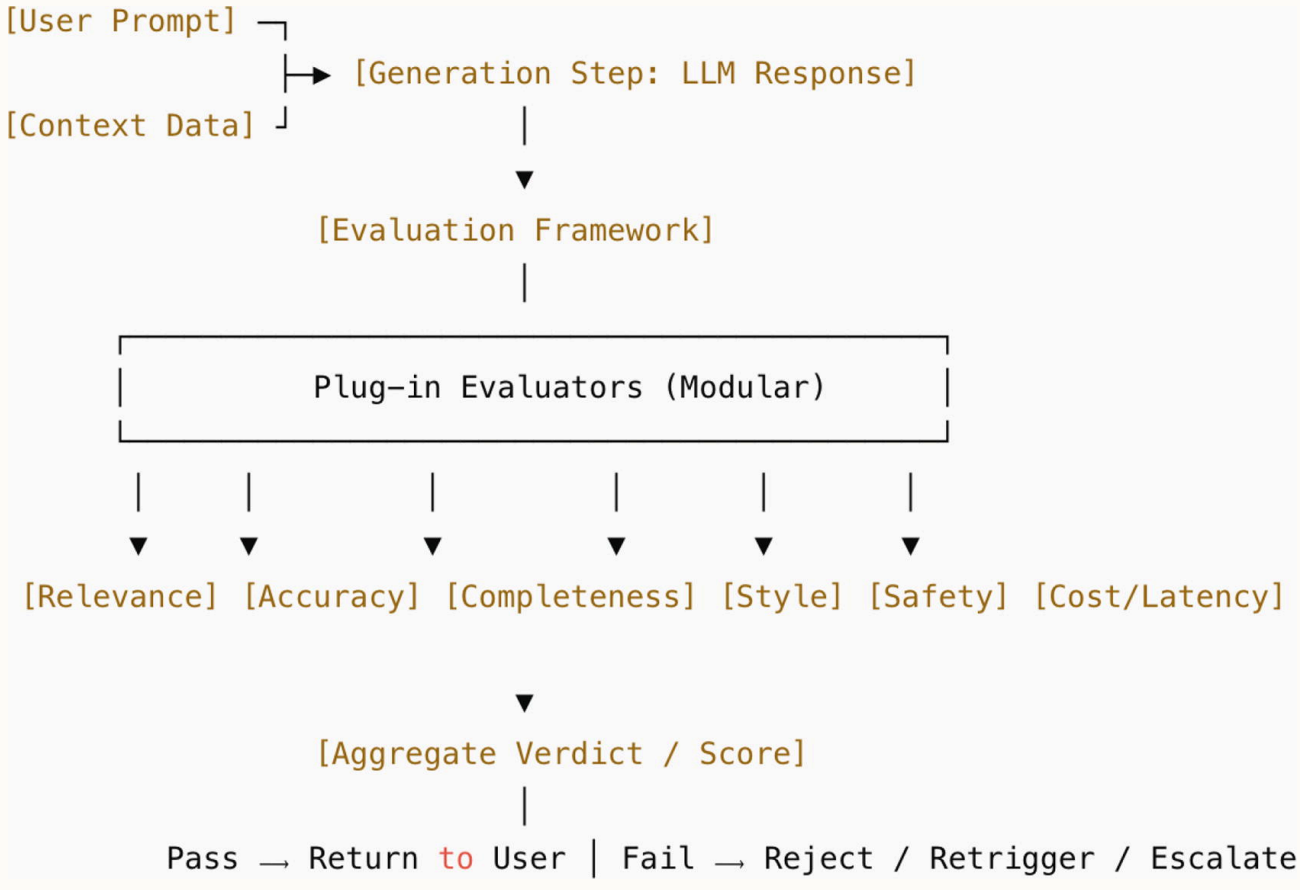


Figure 15-1 Modular evaluation framework for LLM applications

In practice, applications are assessed along multiple quality dimensions. Relevance checks ensure the answer addresses the user's intent and incorporates the supplied context. Accuracy evaluations verify that all factual claims are supported by trusted sources. Completeness audits confirm that no part of the user's request is omitted. Clarity and coherence assessments ensure the response is well-structured and understandable. Style and tone validators enforce branding or regulatory guidelines. Safety and bias detectors guard against harmful or discriminatory content. Finally, performance metrics such as evaluation latency and cost per check are monitored. By composing these evaluators into a continuous quality-gate workflow, teams catch hallucinations, off-topic drift, and unsafe content before it reaches users, safeguarding trust and enabling scalable, responsible AI deployments.

Tools Available for Evaluation and Testing

Table 15-1 summarizes the tools.

Table 15-1 Summary of LLM Evaluation and Testing Tools

Tool/Framework	Language	Key Features	Pros	Limitations
pytest/unittest	Python	Unit/integration testing, mocking, assertions	Widely used, easy to adopt	Not LLM-specific
Hypothesis	Python	Property-based testing with randomized inputs	Great for uncovering edge cases	Steeper learning curve
OpenAI Evals	Python	Task-based evaluation using LLMs	Extensible, community-driven	Requires prompt and task design
LangChain Evaluation Modules	Python	Built-in metrics for relevance, bias, etc.	Deep integration with LangChain apps	Best for LangChain users
Spring AI Evaluator	Java	Relevancy, fact-checking, extensible interface	JUnit-friendly, flexible	Java-only, newer ecosystem
Quarkus LangChain4j Scorer	Java	YAML-driven tests, semantic/AI scoring	Native Quarkus support, detailed CI output	Sparse documentation

Python offers a rich ecosystem of testing and evaluation tools that can be brought to bear on LLM applications. At the unit and integration level, frameworks like pytest and unittest let you mock model calls, simulate user prompts, and assert on outputs or metadata. For behavioral and end-to-end scenarios, property-based testing libraries (e.g., Hypothesis) help uncover edge-case failures by generating diverse inputs automatically. On top of that, dedicated LLM evaluation toolkits—such as OpenAI's evals framework and LangChain's evaluation modules—provide built-in metrics for relevance, factuality, coherence, and bias. You can orchestrate "ground truth versus model" comparisons, run adversarial or red-teaming exercises, and even plug in small, specialized models to flag hallucinations or style-guide violations. Combined with standard CI tooling, these libraries make it straightforward to bake LLM-specific checks into your build and deployment pipelines.

In the Java world, Spring AI brings a first-class abstraction for response evaluation with its simple Evaluator interface. You supply the original user text, any retrieved context, and the model's response, and implementations return pass/fail results or richer scores. Out of the box, Spring AI includes a RelevancyEvaluator that uses an LLM to verify whether an answer truly reflects the supplied context, and a FactCheckingEvaluator that compares each claim against authoritative documents (or even uses lightweight fact-checking models like Minicheck). Because evaluators are decoupled from generation, you can mix and match heavyweight models for dialogue and leaner ones for testing, optimizing cost, and latency. Extensible hooks let you add custom evaluators for style compliance, toxicity screening, bias detection, or domain-specific rules, all invoked through familiar JUnit tests.

For teams building on Quarkus, the quarkus-langchain4j-testing-scorer-junit5 extension provides a Scorer API that evaluates your chat service or RAG retriever against curated samples. By annotating tests with @AiScorer or the ScorerExtension, you can load sample inputs from YAML, choose evaluation strategies (semantic similarity, AI-driven judgments, or your own Java-based checks), and generate per-tag scores and Markdown reports for CI. Langchain4j itself also ships as a stand-alone framework—independent of Quarkus—with similar Scorer and EvaluationStrategy abstractions, but its testing documentation remains sparse. Despite the lean docs, the core concepts carry over: define evaluation samples, plug in built-in or custom strategies, and surface scores in your existing Java test suites to automate relevance, accuracy, style, and safety checks.

Testing and Evaluating Using Spring AI

There are various methods to evaluate LLMs, and one of them is to use another LLM to evaluate the results of the one generating the response. Spring AI provides the Evaluator

interface to evaluate responses.

```
@FunctionalInterface
public interface Evaluator {
   EvaluationResponse evaluate(EvaluationRequest evaluationRequest);
}
```

The input to the evaluate method is the EvaluationRequest, which is defined in the source code as follows. So it takes in the user query along with the relevant document list found through the vector search in our RAG pipeline or any other relevant input augmented to our prompt and the response from the LLM to do the evaluation.

```
public class EvaluationRequest {
   private final String userText;
   private final List<Content> dataList;
   private final String responseContent;
   public EvaluationRequest(String userText, List<Content> dataList, String responseC
ontent) {
      this.userText = userText;
      this.dataList = dataList;
      this.responseContent = responseContent;
   }
  ...
  ...
}
```

There are two implementations of this interface:

1. RelevancyEvaluator

   The RelevancyEvaluator uses an AI model for evaluation. It uses the userText, dataList, and responseContent to formulate and ask the following question to the AI model internally:

```
"Your task is to evaluate if the response for the query
is in line with the context information provided.\n
You have two options to answer. Either YES/ NO.\n
Answer - YES, if the response for the query
is in line with context information otherwise NO.\n
Query: \n {query}\n
Response: \n {response}\n
Context: \n {context}\n
Answer: "
```

2. FactCheckingEvaluator

   The FactCheckingEvaluator assesses the factual accuracy of the AI-generated responses against the provided context. By checking whether each claim is directly supported by the supplied context, this evaluator helps identify and curb hallucinations in AI-generated responses. The evaluator feeds both the "claim" and the "document" into an AI model for assessment. To lower costs and improve efficiency, you can use compact, purpose-built models like Bespoke's Minicheck instead of heavyweight flagship models such as GPT-4. Minicheck is also accessible via Ollama.

The following prompt template is used by the Evaluator for fact checking:

```
Document: {document}
Claim: {claim}
```

Figure 15-2 illustrates this flow.

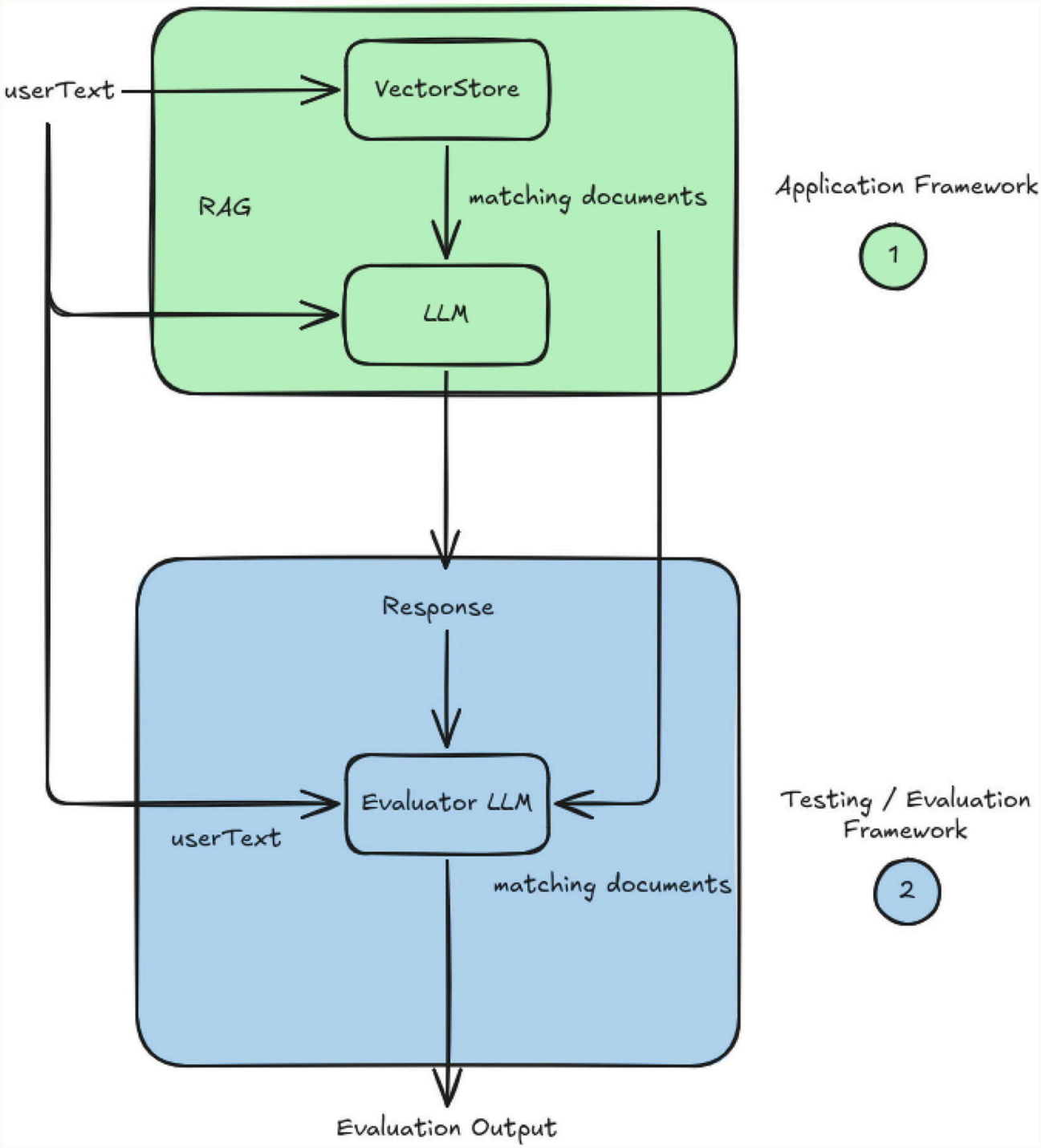


Figure 15-2 Testing/evaluation framework with Spring AI

Now that you understand the technicalities involved, let's get your hands dirty with some implementation to see it in practice. Before you write any tests, you will need a RAG application setup. You can set up any RAG-based application as you prefer, but for the demonstration purposes of this chapter, you will reuse the previous RAG application you built using Spring AI in Chapter 7.

You will JUnit for our test cases and will leverage the default test file already created for you by Spring Boot named DemoApplicationTests.java. First, you will autowire two beans that you previously defined in your Application Boot file, namely, chatModel and vectorStore. These will be used to run the RAG pipeline and gather the response from the model for evaluation.

```
@Autowired
private ChatModel chatModel;
@Autowired
private VectorStore vectorStore;
```

The first test case will be based on RelevancyEvaluator, so you will create a method named testRelevancy and start with getting a response from the model similar to what you tried with the curl request. You are using the ChatClient API to use your chatModel and leverage the vectorStore object through the QuestionAnswerAdvisor RAG utility and pass in our userText query and finally gather the response. The context for the user query is "Airbus A320,2023-11-01,Boston,Las Vegas,BOS,LAS,DL606,14:00,17:30,Cancelled" in the Markdown document for your reference.

```
String userText = "What are the flight details for DL606?";
ChatResponse response = ChatClient.builder(chatModel)
        .build().prompt()
        .advisors(new QuestionAnswerAdvisor(vectorStore))
        .user(userText)
        .call()
        .chatResponse();
```

The next task is to evaluate it for relevancy. Before doing so, you will assert for sanity for the response to not be null and also to print the response for debugging purposes as to what the LLM model responded.

```
Assertions.assertNotNull(response);
String responseText = response.getResult().getOutput().getText();
System.out.println(responseText);
```

Finally, you can evaluate the response by creating a RelevancyEvaluator object and passing in a chatClient object to it. It will use this client API to evaluate the response from earlier.

```
Evaluator relevancyEvaluator = new RelevancyEvaluator(ChatClient.builder(chatMode
l));
```

You then create an EvaluationRequest as discussed earlier with your user query userText and context by retrieving the documents from the VectorStore.

```
List<Document> documents = response.getMetadata().get(QuestionAnswerAdvisor.RE
TRIEVED_DOCUMENTS);
```

```
EvaluationRequest evaluationRequest = new EvaluationRequest(userText, documents,
responseText);
```

The final step is to get the evaluation response by calling the evaluate method on the evaluator object and checking for its truthiness value.

```
EvaluationResponse evaluationResponse = relevancyEvaluator.evaluate(evaluationReq
uest);
assertTrue(evaluationResponse.isPass(), "Response is not relevant to the question");
```

This will successfully validate the test case and validate the relevancy of the response. What about the irrelevance of it? Yes, you can write a negative test case too. For brevity sake, you are writing the test case in the same method, but as a standard practice you can put it in a new test method of its own. You can test for the failure of this case by setting the variable responseText to an irrelevant text like the following, which is not correct, and then rerun the test. The LLM model should be able to check for irrelevancy of the response. The negative test case is shared here:

```
String wrongAnswer = "We are airline company and provide all sort of flights.";
evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswer);
evaluationResponse = relevancyEvaluator.evaluate(evaluationRequest);
assertThat(evaluationResponse.isPass()).isFalse();
```

Let's now run the test case and see the output. Please ignore the skeleton joke output since it comes from the previous example that started with a joke prompt in Chapter 7.

Output:

```
2025-05-12T19:06:57.564Z  WARN 7152 --- [demo] [Test worker] JpaBaseConfigurati
on$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, d
atabase queries may be performed during view rendering. Explicitly configure spring.jp
a.open-in-view to disable this warning
2025-05-12T19:06:58.175Z  INFO 7152 --- [demo] [Test worker] com.example.demo.D
emoApplicationTests : Started DemoApplicationTests in 17.385 seconds (process runni
ng for 19.038)
Sure, here's one for you:
Why don't skeletons fight each other?
They don't have the guts!
The flight details for DL606 on 2023-11-01 are as follows: It was scheduled to depart
from Boston (BOS) to Las Vegas (LAS) at 14:00 and arrive at 17:30. However, the flig
ht was cancelled.
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes
because bootstrap classpath has been appended
```

```
2025-05-12T19:07:05.418Z  INFO 7152 --- [demo] [ionShutdownHook] j.LocalContain
erEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit
'default'
2025-05-12T19:07:05.421Z  INFO 7152 --- [demo] [ionShutdownHook] com.zaxxer.hik
ari.HikariDataSource     : HikariPool-1 - Shutdown initiated...
2025-05-12T19:07:05.425Z  INFO 7152 --- [demo] [ionShutdownHook] com.zaxxer.hik
ari.HikariDataSource     : HikariPool-1 - Shutdown completed.
> Task :test
BUILD SUCCESSFUL in 29s
4 actionable tasks: 2 executed, 2 up-to-date
7:07:05 PM: Execution finished ':test --tests "com.example.demo.DemoApplicationT
ests.testRelevancyEvaluation"'.
```

The test case successfully passed. This shows that the output shared by the LLM model was relevant as verified. You can see the irrelevancy led to the test case passing due to testing it for falseness, ensuring that if in the future the core LLM tends to respond with irrelevant responses, you can detect it. The full function code can be checked here:

```
@Test
void testRelevancyEvaluation() {
    String userText = "What are the flight details for DL606?";
    ChatResponse response = ChatClient.builder(chatModel)
            .build().prompt()
            .advisors(new QuestionAnswerAdvisor(vectorStore))
            .user(userText)
            .call()
            .chatResponse();
    Assertions.assertNotNull(response);
    String responseText = response.getResult().getOutput().getText();
    System.out.println(responseText);
    Evaluator relevancyEvaluator = new RelevancyEvaluator(ChatClient.builder(chatMo
del));
    List<Document> documents = response.getMetadata().get(QuestionAnswerAdvisor.
RETRIEVED_DOCUMENTS);
    EvaluationRequest evaluationRequest = new EvaluationRequest(userText, documen
ts, responseText);
    EvaluationResponse evaluationResponse = relevancyEvaluator.evaluate(evaluationR
equest);
    assertTrue(evaluationResponse.isPass(), "Response is not relevant to the question");
    String wrongAnswer = "We are airline company and provide all sort of flights.";
    evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswer);
    evaluationResponse = relevancyEvaluator.evaluate(evaluationRequest);
    assertThat(evaluationResponse.isPass()).isFalse();
}
```

Now, you will move into the factual evaluation test case. You will define a method named testFactCheckingEvaluation and start with the RAG response collection and invoking the evaluation request as shown here:

```
String userText = "What are the flight details for DL606?";
ChatResponse response = ChatClient.builder(chatModel)
        .build().prompt()
        .advisors(new QuestionAnswerAdvisor(vectorStore))
        .user(userText)
        .call()
        .chatResponse();
Assertions.assertNotNull(response);
String responseText = response.getResult().getOutput().getText();
List<Document> documents = response.getMetadata().get(QuestionAnswerAdvisor.RE
TRIEVED_DOCUMENTS);
EvaluationRequest evaluationRequest = new EvaluationRequest(userText, documents,
responseText);
```

The next step is to initialize an instance of the FactCheckingEvaluator with a chat model of your choice to do the fact evaluation and run the evaluate method call on it to collect the evaluation response:

```
Evaluator factCheckingEvaluator = new FactCheckingEvaluator(ChatClient.builder(cha
tModel));
EvaluationResponse evaluationResponse = factCheckingEvaluator.evaluate(evaluation
Request);
assertThat(evaluationResponse.isPass()).isTrue();
```

This will check for the fact details and evaluate with a true response. Also let's evaluate the opposite case where you set some factual information that is not correct in this context and see it failing. This could be part of another test case since each unique condition should be a test case. For brevity, I am sharing both in a single test case:

```
String wrongAnswer = "DL606 is taking off from New York.";
evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswer);
evaluationResponse = factCheckingEvaluator.evaluate(evaluationRequest);
assertThat(evaluationResponse.isPass()).isFalse();
```

This will fail and will pass our test case since the DL606 flight is taking off from Boston instead, and the fact is not correct. Let's run this and check the output:

Output:

```
2025-05-12T19:02:17.227Z  WARN 6159 --- [demo] [Test worker] JpaBaseConfiguratio
n$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, dat
abase queries may be performed during view rendering. Explicitly configure spring.jpa.
open-in-view to disable this warning
2025-05-12T19:02:18.045Z  INFO 6159 --- [demo] [Test worker] com.example.demo.
DemoApplicationTests    : Started DemoApplicationTests in 19.436 seconds (process ru
nning for 21.301)
Why don't skeletons fight each other?
They don't have the guts.
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes
because bootstrap classpath has been appended
> Task :test
2025-05-12T19:02:26.742Z  INFO 6159 --- [demo] [ionShutdownHook] j.LocalContain
erEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit
'default'
2025-05-12T19:02:26.747Z  INFO 6159 --- [demo] [ionShutdownHook] com.zaxxer.hi
kari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2025-05-12T19:02:26.751Z  INFO 6159 --- [demo] [ionShutdownHook] com.zaxxer.hik
ari.HikariDataSource       : HikariPool-1 - Shutdown completed.
BUILD SUCCESSFUL in 33s
4 actionable tasks: 2 executed, 2 up-to-date
7:02:27 PM: Execution finished ':test --tests "com.example.demo.DemoApplicationTes
ts.testFactCheckingEvaluation"'.
```

The full function code can be checked here:

```
@Test
void testFactCheckingEvaluation() {
    String userText = "What are the flight details for DL606?";
    ChatResponse response = ChatClient.builder(chatModel)
            .build().prompt()
            .advisors(new QuestionAnswerAdvisor(vectorStore))
            .user(userText)
            .call()
            .chatResponse();
    Assertions.assertNotNull(response);
    String responseText = response.getResult().getOutput().getText();
    List<Document> documents = response.getMetadata().get(QuestionAnswerAdvisor.
RETRIEVED_DOCUMENTS);
    EvaluationRequest evaluationRequest = new EvaluationRequest(userText, documen
ts, responseText);
    Evaluator factCheckingEvaluator = new FactCheckingEvaluator(ChatClient.builder(c
hatModel));
    EvaluationResponse evaluationResponse = factCheckingEvaluator.evaluate(evaluati
onRequest);
```

```
    assertThat(evaluationResponse.isPass()).isTrue();
    String wrongAnswer = "DL606 is taking off from New York.";
    evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswer);
    evaluationResponse = factCheckingEvaluator.evaluate(evaluationRequest);
    assertThat(evaluationResponse.isPass()).isFalse();
  }
```

Here is the final code for the test case file:

File: ./src/test/java/com/example/demo/DemoApplicationTests.java

```
package com.example.demo;
import org.junit.jupiter.api.Assertions;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.document.Document;
import org.springframework.ai.evaluation.*;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.beans.factory.annotation.Autowired;
import org.junit.jupiter.api.Test;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.boot.test.context.SpringBootTest;
import java.util.List;
import static org.assertj.core.api.Assertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertTrue;
@SpringBootTest
class DemoApplicationTests {
    @Autowired
    private ChatModel chatModel;
    @Autowired
    private VectorStore vectorStore;
    @Test
    void testRelevancyEvaluation() {
        String userText = "What are the flight details for DL606?";
        ChatResponse response = ChatClient.builder(chatModel)
                .build().prompt()
                .advisors(new QuestionAnswerAdvisor(vectorStore))
                .user(userText)
                .call()
                .chatResponse();
        Assertions.assertNotNull(response);
        String responseText = response.getResult().getOutput().getText();
        System.out.println(responseText);
```

```
        Evaluator relevancyEvaluator = new RelevancyEvaluator(ChatClient.builder(chat
Model));
        List<Document> documents = response.getMetadata().get(QuestionAnswerAdvis
or.RETRIEVED_DOCUMENTS);
        EvaluationRequest evaluationRequest = new EvaluationRequest(userText, docum
ents, responseText);
        EvaluationResponse evaluationResponse = relevancyEvaluator.evaluate(evaluatio
nRequest);
        assertTrue(evaluationResponse.isPass(), "Response is not relevant to the questio
n");
        String wrongAnswer = "We are airline company and provide all sort of flights.";
        evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswe
r);
        evaluationResponse = relevancyEvaluator.evaluate(evaluationRequest);
        assertThat(evaluationResponse.isPass()).isFalse();
    }
    @Test
    void testFactCheckingEvaluation() {
        String userText = "What are the flight details for DL606?";
        ChatResponse response = ChatClient.builder(chatModel)
                .build().prompt()
                .advisors(new QuestionAnswerAdvisor(vectorStore))
                .user(userText)
                .call()
                .chatResponse();
        Assertions.assertNotNull(response);
        String responseText = response.getResult().getOutput().getText();
        List<Document> documents = response.getMetadata().get(QuestionAnswerAdvis
or.RETRIEVED_DOCUMENTS);
        EvaluationRequest evaluationRequest = new EvaluationRequest(userText, docum
ents, responseText);
        Evaluator factCheckingEvaluator = new FactCheckingEvaluator(ChatClient.builder
(chatModel));
        EvaluationResponse evaluationResponse = factCheckingEvaluator.evaluate(evalua
tionRequest);
        assertThat(evaluationResponse.isPass()).isTrue();
        String wrongAnswer = "DL606 is taking off from New York.";
        evaluationRequest = new EvaluationRequest(userText, documents, wrongAnswe
r);
        evaluationResponse = factCheckingEvaluator.evaluate(evaluationRequest);
        assertThat(evaluationResponse.isPass()).isFalse();
    }
}
```

Conclusion

Testing and evaluating LLM applications is no longer an optional afterthought—it's a pillar of responsible, scalable AI development. By treating evaluation as a stand-alone stage, you can mix lightweight or purpose-built models for checks, dramatically reducing both latency and cost compared to running a flagship model for every test. Whether you adopt Python-based evaluation frameworks or Spring AI's Evaluator abstractions, the critical steps remain the same: define clear success criteria for each quality dimension, automate checks in your CI/CD workflows, and integrate telemetry and alerting to catch regressions in production.

As LLM technology and enterprise requirements continue to evolve, your evaluation processes must stay ahead. Regularly revisit your metrics and thresholds, update your test suites with new adversarial or compliance scenarios, and leverage observability data to refine prompts and model configurations. With a robust evaluation and testing strategy in place, you transform LLM unpredictability from a liability into a predictable, manageable aspect of your software delivery lifecycle—empowering you to ship innovative AI features with confidence and control.

For further reading and information, please refer to the following:

- OpenAI evals: Offers a structured framework to benchmark and compare LLM outputs using custom or community-contributed evaluation tasks.
- LangChain evaluation modules: Provide built-in tools for assessing LLM chains across dimensions such as accuracy, coherence, and latency—ideal for integrated LangChain pipelines.
- Spring AI Evaluator docs: Introduce abstractions to plug evaluation steps directly into Spring-based workflows, making it easier to monitor and manage quality in Java-centric AI applications.

The next chapter covers Gen AI in the cloud.

Previous chapter

< 14. Seeing Is Believing: Multimodal LLMs and Image Hacking

Next chapter

16. Cloud Power-Ups: Bedrock, Vertex, and Azure OpenAI >